



Pontifícia Universidade Católica do Rio Grande do Sul
Confiabilidade e Segurança de Software
98G08-04
Lista de Exercícios
Códigos de Autenticação de Mensagem (MACs)
(Com Respostas)

Prof. Iaçanã Ianiski Weber

Exercícios adaptados de Paar, Pelzl e Güneysu, *Understanding Cryptography*, 2ª ed., Springer, 2024 (Capítulo 13).

Observação: esta versão contém os enunciados e uma proposta de solução. Em itens discursivos, respostas equivalentes podem ser igualmente corretas.

Exercício 1

Queremos contrastar dois protocolos que protegem uma mensagem x : um baseado em MAC e outro em assinatura digital. Em ambos, o remetente calcula e envia y , onde (Livro, Problema 13.1)

$$\text{Protocolo A: } y = e_{k_1}[x \parallel h(k_2 \parallel x)] \quad \text{Protocolo B: } y = e_k[x \parallel \text{sig}_{k_{pr}}(h(x))]$$

(k_1, k_2 são chaves simétricas secretas conhecidas só pelas duas partes; k_{pr}/k_{pub} é o par assimétrico do remetente.)

- Descreva, **passo a passo**, o que o receptor faz ao receber y em cada protocolo.
- Quais **serviços de segurança** cada protocolo oferece?

(a) Protocolo A (autenticação simétrica):

- Decifra a camada externa: $x' \parallel t' = d_{k_1}(y)$. Como a saída do hash tem tamanho fixo conhecido (ex.: 256 bits), separa os últimos bits como a *tag* recebida t' e o restante como a mensagem x' .
- Recalcula o MAC com a chave k_2 que também possui: $t^* = h(k_2 \parallel x')$.
- Compara t^* com t' (idealmente em tempo constante). Se iguais, aceita x' como íntegra e autêntica; senão, rejeita.

Protocolo B (assinatura digital):

- Decifra: $x' \parallel s' = d_k(y)$. Separa a mensagem x' e a assinatura s' (tamanho conhecido).
- Recalcula o hash da mensagem recebida: $z^* = h(x')$.
- Verifica a assinatura com a *chave pública do remetente*: $\text{ver}_{k_{pub}}(s', z^*)$. É essencial confrontar s' com $z^* = h(x')$ da mensagem recebida (não basta “abrir” a assinatura). Se válida, aceita; senão, rejeita.

(b) Ambos oferecem **confidencialidade** (camada e_{k_1}/e_k), **integridade** e **autenticação de origem**. A diferença essencial é o **não repúdio**, que só o Protocolo B oferece. Razão: o não repúdio exige *assimetria* entre gerar e verificar a prova de autoria. Em B, a assinatura usa a chave privada k_{pr} , que *só o remetente* possui (e qualquer um verifica com k_{pub}); ela vincula a mensagem unicamente ao remetente, que não pode negá-la diante de um árbitro. Em A, o MAC usa a chave *simétrica* k_2 , compartilhada: como emissor e receptor podem gerar $h(k_2 \parallel x)$, eles são indistinguíveis e o emissor sempre pode alegar que foi o outro quem forjou a mensagem. MACs simétricos, por construção, nunca dão não repúdio.

Notas para o professor: (i) o não repúdio de B só vale perante terceiros se k_{pub} estiver autenticada (certificado/PKI). (ii) O construtor $h(k_2||x)$ é de *prefixo secreto*: em hash Merkle–Damgård sofre *length extension* (ver Ex. 3); na prática usar-se-ia HMAC/KMAC. Isso não altera a classificação de serviços.

Exercício 2

Para funções hash é crucial ter uma saída longa (p.ex. 256 bits) para frustrar ataques baseados no paradoxo do aniversário. No entanto, comprimentos bem menores — p.ex. **128 bits** — costumam ser suficientes para **MACs**. (Livro, Problema 13.2)

- (a) Suponha que a mensagem x trafega em claro junto com seu MAC, isto é, o par $(x, \text{MAC}_k(x))$. Esclareça **exatamente** o que Oscar (que não conhece k) precisa fazer para forjar uma mensagem aceita pelo receptor.
- (b) Por que o paradoxo do aniversário **não** ajuda Oscar nesse cenário? Compare o custo do ataque (*online* vs. *offline*) e conclua qual o nível de segurança de uma *tag* de MAC de 128 bits comparada a um hash de 128 bits.

(a) Oscar quer produzir um par (x', m') com x' nova e $m' = \text{MAC}_k(x')$, de modo que o receptor (que tem k) aceite. Sem a chave, ele tem duas opções de força bruta: (i) adivinhar a chave k , custo $\approx 2^{|k|}$; ou (ii) adivinhar diretamente a *tag* de uma x' escolhida, com probabilidade de acerto 2^{-n} por palpite (n = tamanho da *tag*). O ponto **crucial**: para saber se um palpite m' está certo, Oscar **precisa submeter (x', m') ao verificador**, pois sozinho ele não consegue calcular $\text{MAC}_k(\cdot)$. Logo a falsificação exige $\approx 2^n$ tentativas **online**.

(b) O paradoxo do aniversário *não* se aplica porque ele pressupõe que o atacante consiga *computar a função por conta própria* e reconhecer colisões:

- **Hash (colisão) = offline.** h é pública e sem chave; Oscar calcula $h(\cdot)$ na própria máquina, em paralelo e em silêncio. O aniversário garante uma colisão após $\approx 2^{n/2}$ avaliações. Segurança efetiva $\approx 2^{n/2}$.
- **MAC (forja) = online.** Sem k , Oscar não computa MAC_k sozinho nem reconhece colisões offline; cada tentativa exige uma consulta real ao detentor da chave. Isso é lento, não paralelizável fora do sistema, controlável e *detectável* (uma rajada de MACs inválidos dispara *rate-limiting*/bloqueio). Segurança $\approx 2^n$.

Conclusão: a chave secreta transfere a verificação para algo que *só o defensor* pode fazer, empurrando Oscar de um ataque offline barato ($2^{n/2}$) para um ataque online caro e detectável (2^n). Por isso uma *tag* de MAC de **128 bits** oferece $\approx 2^{128}$ de segurança, enquanto um **hash** de 128 bits oferece apenas $\approx 2^{64}$ contra colisão (daí hashes pedirem ≥ 256 bits). De forma geral, a segurança do MAC é $\min(2^{\text{tag}}, 2^{|k|})$.

Nota para o professor: o valor $\approx 2^n$ pressupõe um MAC *sem fraqueza estrutural*. Se houver, valem ataques específicos que o aniversário *volta* a habilitar — *length extension* no prefixo secreto, colisão no sufixo secreto (Ex. 3), falsificação existencial no CBC-MAC puro (Ex. 5) — e é por isso que se usam HMAC/KMAC e CMAC.

Exercício 3

Considere MACs construídos a partir de uma função hash H criptograficamente forte baseada na construção de **Merkle–Damgård**. Mostre os **dois ataques principais** contra as construções ingênuas de prefixo secreto e sufixo secreto: (Livro, Problema 13.7)

$$\text{MAC}_k(x) = H(k \parallel x) \text{ (prefixo secreto)}$$

$$\text{MAC}_k(x) = H(x \parallel k) \text{ (sufixo secreto)}$$

- (a) **Prefixo secreto:** descreva como o atacante forja um MAC válido para uma mensagem *estendida*, sem conhecer k . Que nome tem esse ataque?
- (b) **Sufixo secreto:** descreva como o atacante explora uma colisão do hash para forjar um MAC válido.
- (c) Por que o **SHA-3** (construção esponja) não sofre o ataque do item (a)?

(a) Prefixo secreto — length extension. Numa função Merkle–Damgård, a saída $m = H(k \parallel x)$ é *exatamente o estado interno de encadeamento* após absorver $k \parallel x$ (com seu *padding*). Conhecendo apenas (x, m) — e os parâmetros públicos IV , tamanho de bloco, regra de *padding* e o comprimento $|k|$ — o atacante usa m como estado inicial e **continua** a iteração, anexando um sufixo arbitrário s . Obtém

$$m' = H(\underbrace{k \parallel x \parallel \text{pad}(k \parallel x)}_{\text{prefixo original} + \text{“glue”}} \parallel s)$$

isto é, um MAC válido para a mensagem estendida $x_{\text{forj}} = x \parallel \text{pad}(k \parallel x) \parallel s$, **sem jamais conhecer k** . É a **falsificação por extensão de comprimento** (*length extension attack*); o sufixo s poderia ser, p. ex., um apêndice malicioso a um contrato.

(b) Sufixo secreto — colisão. Aqui o *length extension* não serve (k vem por último). Em vez disso, o atacante busca **offline** (sem chave) uma colisão $h(x_1) = h(x_2)$ com $x_1 \neq x_2$ (custo $\approx 2^{n/2}$ pelo aniversário). Como o estado de encadeamento após x_1 e x_2 é o mesmo, e k é absorvido *depois* de forma idêntica nas duas, segue

$$m = h(x_1 \parallel k) = h(x_2 \parallel k) \Rightarrow \text{MAC}_k(x_1) = \text{MAC}_k(x_2).$$

O atacante obtém o MAC legítimo de x_1 (uma única consulta) e apresenta (x_2, m) , que será aceito. As duas mensagens podem ser, p. ex., duas versões de um contrato.

(c) SHA-3 (esponja) é imune ao length extension. O estado tem largura $b = r + c$ (*rate* + *capacity*); a saída expõe apenas bits lidos do *rate*, e os c bits de **capacidade nunca são revelados**. Para continuar a hash, o atacante precisaria do estado completo, mas recuperar os c bits ocultos custa $\approx 2^c$ (p. ex. 2^{512} no SHA3-256), inviável. Por isso o **KMAC** ($\approx \text{SHA3}(k \parallel x)$) é seguro *sem* o envelope do HMAC, e o NIST o padronizou diretamente sobre o Keccak. Com SHA-1/SHA-2 (Merkle–Damgård), o aninhamento externo chaveado do **HMAC** = $H((k \oplus \text{opad}) \parallel H((k \oplus \text{ipad}) \parallel x))$ “fecha” a saída interna e anula o ganho do *length extension*.

Notas para o professor: (i) a mensagem forjada em (a) inclui o *glue padding* $\text{pad}(k \parallel x)$ embutido no meio — ele é parte essencial da forja, e reconstruí-lo exige conhecer $|k|$ (senão, itera-se sobre os poucos comprimentos plausíveis). (ii) Em (b), para o argumento ser exato a colisão deve ter o *mesmo comprimento em blocos* (colisão de estado interno), de modo que o alinhamento de k e do *padding* final seja idêntico nas duas mensagens.

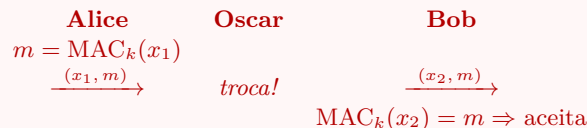
Exercício 4

MACs também são, em princípio, vulneráveis a ataques de colisão. (Livro, Problema 13.5)

- (a) Suponha que Oscar encontrou uma colisão entre duas mensagens, isto é, $\text{MAC}_k(x_1) = \text{MAC}_k(x_2)$ com $x_1 \neq x_2$. Considere uma troca de mensagens autenticadas (**sem** encriptação) entre Alice e Bob. Mostre o que Oscar precisa fazer para explorar a colisão, lembrando que ele **não** possui a chave k .
- (b) Embora o paradoxo do aniversário ainda possa ser usado para construir colisões, por que é muito mais difícil construí-las na prática para MACs do que para funções hash? Em consequência, que segurança oferece um MAC de saída de 128 bits comparado a uma função hash de saída de 128 bits?

(a) Como Oscar **não** tem k , ele não consegue gerar nenhuma *tag* válida sozinho; o único recurso é **reaproveitar** uma *tag* legítima. O ataque (substituição por colisão):

1. Oscar induz Alice a enviar uma das mensagens da colisão, x_1 (tipicamente a *benigna* — p. ex., “transferir R\$ 10”); Alice calcula $m = \text{MAC}_k(x_1)$ e transmite (x_1, m) .
2. Como não há encriptação, Oscar lê x_1 e *copia a tag m em claro.*
3. Oscar substitui a mensagem mantendo a *mesma tag*: injeta (x_2, m) para Bob, onde x_2 é a versão fraudulenta (p. ex., “transferir R\$ 10000”).
4. Bob recalcula $\text{MAC}_k(x_2)$; pela colisão, o resultado é m — exatamente o valor recebido. Bob **aceita** x_2 como autêntica.



A forja é bem-sucedida *sem* Oscar conhecer k ; ela depende de Alice enviar a parceira x_1 da colisão, e a ausência de encriptação é o que lhe permite ler/copiar a *tag*.

(b) Para construir uma colisão por aniversário são necessárias $\approx 2^{n/2}$ avaliações da função. Para um **hash**, Oscar faz isso *offline*, sozinho, pois a função é sem chave. Para um **MAC**, ele não tem k , então teria de *enganar Alice/Bob* para que computassem essa quantidade enorme de MACs — um ataque *online*, lento, controlável e detectável, inviável para n grande. Logo, um MAC de 128 bits oferece $\approx 2^{128}$ de segurança (colisões não aplicáveis na prática), enquanto um hash de 128 bits oferece apenas $\approx 2^{64}$.

Exercício 5

Estudamos um ataque adaptativo de texto-claro escolhido contra o **CBC-MAC**, mostrando que ele pode ser inseguro. Assuma um IV todo-zero e que Oscar tem acesso a um oráculo $\mathcal{O}$ que calcula o MAC de mensagens de tamanho arbitrário com uma chave k desconhecida. Cada bloco x_i tem a largura do bloco da cifra. (Livro, Problema 13.8)

- (a) O objetivo de Oscar é construir um MAC válido para a mensagem $X' = x_1 x_2 x_3$ **sem** pedir $\text{MAC}(X')$ ao oráculo. Ele primeiro pede o MAC de $X = x_1 x_2$ e obtém m . Que **outra mensagem** ele precisa enviar ao oráculo para passar a conhecer $\text{MAC}(X')$?

- (b) Explique por que o ataque funciona, isto é, qual é o aspecto problemático do CBC-MAC.
- (c) O ataque ainda é possível se for usado o esquema **CMAC**? Justifique sua resposta.

Lembre o CBC-MAC com $IV = 0$: $t_1 = e_k(x_1)$, $t_i = e_k(x_i \oplus t_{i-1})$, e MAC = último bloco.

(a) Oscar faz **duas** consultas. Primeiro pede $m = \mathcal{O}(x_1 x_2)$, internamente $m = e_k(x_2 \oplus e_k(x_1))$. A *segunda mensagem* é o **bloco único** $Q = x_3 \oplus m$. Como $IV = 0$, o oráculo devolve

$$\mathcal{O}(Q) = e_k(Q \oplus 0) = e_k(x_3 \oplus m).$$

Mas a cadeia de $X' = x_1 x_2 x_3$ daria justamente $t_3 = e_k(x_3 \oplus t_2) = e_k(x_3 \oplus m)$. Logo

$$\text{MAC}(X') = e_k(x_3 \oplus m) = \mathcal{O}(Q),$$

e o par $(X', \mathcal{O}(Q))$ é uma falsificação válida, sem nunca consultar X' .

(b) O ataque funciona porque, no CBC-MAC puro com IV fixo, **a tag é o próprio valor de encadeamento** exposto na saída, e *nada amarra a tag ao comprimento* da mensagem. Conhecendo $m = t_2$, “continuar a cadeia por fora” com um bloco x_3 equivale a calcular $e_k(x_3 \oplus m)$ — que é o MAC da mensagem de um bloco $x_3 \oplus m$ (pois $IV = 0$ faz a primeira rodada ser e_k da própria entrada). É a clássica **falsificação existencial por extensão**: o CBC-MAC puro só é seguro para mensagens de comprimento *fixo* e previamente acordado.

(c) Não, o ataque **não** funciona contra o **CMAC**. No CMAC, deriva-se de k uma subchave secreta k_1 (a partir de $e_k(0)$) que é somada (XOR) *apenas no último bloco* antes da cifragem final. Consequências:

- A *tag* de $x_1 x_2$ passa a ser $m = e_k(x_2 \oplus t_1 \oplus k_1)$, que **não** é o valor de encadeamento “cru” $t_2 = e_k(x_2 \oplus t_1)$. Oscar não consegue recuperar t_2 a partir de m (precisaria inverter e_k).
- Para $X' = x_1 x_2 x_3$, o último bloco é x_3 , então $\text{MAC}(X') = e_k(x_3 \oplus t_2 \oplus k_1)$, com t_2 sem k_1 (pois agora o 2º bloco não é o último). E a consulta de $Q = x_3 \oplus m$ devolveria $e_k(x_3 \oplus m \oplus k_1)$, sem relação com o alvo.

Como k_1 é secreto e o tratamento do último bloco depende do comprimento, a *tag* fica **amarrada ao comprimento** e não pode ser reaproveitada como valor de encadeamento — o que elimina a falsificação por extensão. É exatamente a fraqueza que o CMAC foi projetado para corrigir.

Nota para o professor: a segunda consulta é uma mensagem de um *único bloco* $x_3 \oplus m$, não o par (x_3, m) . Isso depende de $IV = 0$; com um $IV \neq 0$ conhecido, consultar-se-ia $Q = x_3 \oplus m \oplus IV$.

Exercício 6

Vamos analisar uma construção de MAC *ad hoc*, eficiente mas criptograficamente fraca. A mensagem $X = x_1 \parallel x_2 \parallel \dots \parallel x_z$ tem z blocos independentes, cada x_i de 8 bits, que entram numa função de compressão

$$c_i = h(c_{i-1}, x_i) = c_{i-1} \oplus x_i, \quad \text{com } c_i, x_i \text{ de 8 bits,}$$

e o MAC é calculado a partir da última saída c_z :

$$\text{MAC}_k(X) \equiv c_z + k \pmod{2^8}, \quad k \text{ é uma chave compartilhada de 64 bits.}$$

- (a) Descreva como a parte *efetiva* da chave k pode ser calculada com apenas **uma** mensagem conhecida X (e seu MAC).

- (b) Realize o ataque e determine k para: $X = \text{HELLO ALICE!}$ (string ASCII, um bloco de 8 bits por caractere, *incluindo o espaço*), $c_0 = 1111\ 1111_2$ e $\text{MAC}_k(X) = 1001\ 1101_2$. Mostre o cálculo.
- (c) Qual é o comprimento **efetivo** da chave k ?

(a) Como $c_i = c_{i-1} \oplus x_i$ e o XOR é associativo e comutativo, desenrolando a cadeia:

$$c_z = c_0 \oplus x_1 \oplus x_2 \oplus \cdots \oplus x_z,$$

que depende *apenas* de dados públicos (c_0 e a mensagem X) — **não depende da chave**. De posse de um par conhecido $(X, \text{MAC}_k(X))$, o atacante calcula c_z sozinho e isola k invertendo a soma modular:

$$k \equiv \text{MAC}_k(X) - c_z \pmod{2^8}.$$

Uma única mensagem conhecida basta (sem busca!) para recuperar os 8 bits efetivos de k , com os quais ele forja o MAC de *qualquer* mensagem nova.

(b) Códigos ASCII de HELLO ALICE! (12 blocos, incluindo o espaço 0x20):

H=48 E=45 L=4C L=4C O=4F _=20 A=41 L=4C I=49 C=43 E=45 !=21

O XOR de todos os 12 bytes é 0x01, logo

$$c_z = c_0 \oplus \left(\bigoplus_i x_i \right) = 0xFF \oplus 0x01 = 0xFE = 254.$$

Recuperando a chave (módulo 2^8):

$$k \equiv \text{MAC} - c_z = 0x9D - 0xFE = 157 - 254 = -97 \equiv 159 \pmod{256} = 0x9F = 1001\ 1111_2.$$

Conferência: $(c_z + k) \bmod 256 = (254 + 159) \bmod 256 = 413 - 256 = 157 = 0x9D = 1001\ 1101_2 = \text{MAC}$. ✓

(c) O comprimento **efetivo** é de apenas **8 bits**. Embora k tenha 64 bits nominais, a operação $(c_z + k) \bmod 2^8$ só usa os 8 bits menos significativos de k ; os 56 bits superiores nunca influenciam a *tag*. O espaço de chave útil é $2^8 = 256$ — e o ataque sequer precisa de busca exaustiva, pois uma única equação fornece k diretamente.